Miguel Core Unified Daemon

Standalone printable deep-dive dossier

Miguel Core Unified Daemon

Early systems had separate daemons, command scripts, and services. Miguel Core consolidated the control surface into a local Python daemon that could coordinate chat, streaming, voice, memory, model tiers, orchestration, state, and tool endpoints.

- This shows system-level building rather than model use: interfaces, state, memory, tools, routing, health, orchestration, and operating boundaries were treated as one control plane.
- Local experimental control plane, not a hardened commercial security product.

What I had in mind

Miguel Core came from a very practical frustration: I had too many strong pieces living as separate scripts and services. I wanted one local control layer that could make the system observable, routable and easier to reason about.

What I built

- Central file `miguel_core.py` documents the consolidation goal and exposes the route surface. `orchestra.py` implements async worker delegation to Opus, Codex, GPT-style workers through a subprocess layer with timeouts, cancellation, status, and stats.

Mechanism detail

- Miguel Core is best explained as a local agent operating layer. It consolidates chat, streaming, voice, state, memory, prediction, model routing, orchestration and tool endpoints into one observable control plane.
- The route surface matters because it shows the work moved from isolated scripts toward a system with health, state, conversation persistence and worker delegation.
- The surrounding startup and handover documents are part of the evidence: they define boot checks, operating modes, quality anchors, autonomy rings and state probes.

Architecture

- One daemon replaces fragmented service surfaces.
- Routes cover chat/streaming, voice, state, vault, knowledge, prediction, cascade, agents, metacognition, orchestra, macOS/tool endpoints, and conversations.
- Imports connect memory broker, CASS pipeline, vault watcher, prediction, metacognition, state graph, orchestra, and macOS bridge.
- Startup/handover scripts define model routing, state probes, boot checks, and autonomy rings.

Evidence cluster

- Miguel Core daemon source documenting the consolidation of earlier daemon/command surfaces into one local control plane.
- Route map covering chat, streaming, voice, state, vault, knowledge, prediction, cascade, agents, orchestra, macOS/tool control, conversations, and health surfaces.

- Orchestra worker module implementing delegated async agent tasks, timeouts, cancellation, status, and result capture.
- Startup and handover documents showing boot probes, routing policies, quality gates, operating modes, and autonomy rings.

Claim-to-evidence map

The public version should be strong because it is bounded, not because it overclaims.

Claim	Evidence	Boundary
This shows system-level building rather than model use: interfaces, state, memory, tools, routing, health, orchestration, and operating boundaries were treated as one control plane.	Miguel Core daemon source documenting the consolidation of earlier daemon/command surfaces into one local control plane.; Route map covering chat, streaming, voice, state, vault, knowledge, prediction, cascade, agents, orchestra, macOS/tool control, conversations, and health surfaces.	Local experimental control plane, not a hardened commercial security product.
The work is valuable because it shows mechanism, not surface polish.	One daemon replaces fragmented service surfaces.; Routes cover chat/streaming, voice, state, vault, knowledge, prediction, cascade, agents, metacognition, orchestra, macOS/tool endpoints, and conversations.	Fresh public demos or benchmarks would be the next proof layer.

Senior-level signal

This shows system-level building rather than model use: interfaces, state, memory, tools, routing, health, orchestration, and operating boundaries were treated as one control plane.

- Architecture consolidation: recognizing fragmentation and pulling many surfaces into one controllable layer.
- Operational maturity: boot probes and handover docs show that the system was meant to be run, not only admired.
- Boundary maturity: autonomy rings separate what the system may do from what must remain user-authorized.

Skeptical reviewer questions

- What is actually built here, and what is still design? Answer: the status and boundary are stated directly inside the Miguel Core Unified Daemon case.
- Which private evidence is intentionally not public? Answer: local paths, credentials, private channel details and confidential raw material are excluded.
- Why is this relevant? Answer: the case shows a reusable component for agents, tool use, memory, routing, validation or high-stakes governance.
- What would be the next stronger proof? Answer: synthetic demo, audit trace, benchmark or external review, depending on the case.

Lessons and boundaries

Local experimental control plane, not a hardened commercial security product.

- Agent reliability depends on process state, permissions, memory, model availability, and user intent.
- A route existing is not the same as a route being production-hardened.
- Local-first systems still need explicit observability and failure boundaries.

- Autonomy rings are a useful design language for separating safe actions from user-only actions.

Next hardening / next project step

- Create a public demo mode with confidential access details disabled.
- Add endpoint-level permission metadata.
- Add machine-readable health reports for all major modules.
- Shrink the public API surface into documented stable interfaces.
- Expose a public demo route map with all private actions replaced by safe stubs.
- Add endpoint metadata: required permission, data touched, model used, timeout and audit trace.
- Publish a synthetic health report that proves every route can declare its current readiness state.